

학부 트랙 아이디어 지도

첫 프레임 한 장과 16스텝짜리 로봇 관절 명령으로 미래 영상을 만드는 과제. 학부 트랙 6개 팀은 거의 전부 “모델이 액션을 무시한다”는 같은 문제를 만났고, 서로 다른 다섯 가지 방법으로 그것을 풀었다.

대상 undergrad/team1 - team8 제출 팀 6팀 (team2 · team5 폴더 비어 있음)
근거 제출 README · config · 코드 · 검사보고서 · 방법론 그림 문서 정리일 2026-09-07

요약

먼저 알아야 할 세 가지

01

학부 트랙의 공통 적은 “조건 무시”였다

배점 0.4짜리 Action 성분을 잡으려면 모델이 액션을 실제로 반영해야 하는데, team6은 액션을 통째로 바꿔도 오차가 1.84% 밖에 안 오른다는 걸 실측했다. 여섯 팀 모두 여기에 각자의 처방을 냈다.

02

처방은 다섯 갈래로 갈렸다

학습 태스크를 바꾸거나(team1·3), 무시 불가능한 통로를 쓰거나(team4), 전용 통로를 신설하거나(team6), 자체 심판을 세우거나(team7), 추론에서 조건을 밀어붙이는(team8) 다섯 가지다.

03

1위와 3위의 코드가 사실상 같다

team1↔team3의 구조 유사도가 0.783인 반면 다른 모든 쌍은 0.15–0.26이다. 학습 창 수·홀드아웃 24장면·하이퍼파라미터 30개가 전부 일치한다. 여섯 팀이 아니라 다섯 개의 아이디어로 읽어야 한다.

한눈에 보는 6팀

팀	순위	백본	“액션 무시” 처방	추론 특징	점수
team1	Private 1위	Cosmos3-Nano (동결)	학습 태스크를 번갈아 — 홀수 스텝엔 액션도 맞히게	30 step · 단일 생성	0.129782
team3	Private 3위	Cosmos3-Nano (동결)	동일 (team1과 사실상 같은 코드) + 화질 증강	30 step · VLM 캡션 조건	0.1373
team4	4위	Wan2.2 TI2V-5B (동결)	AdaLN-Zero — 무시할 수 없는 통로 + 움직임 가중 손실	20 step · 5개 생성 후 메도이드	0.147246
team6	6위	DynamiCrafter_512	액션 전용 cross- attention 통로 신설 (전체의 0.08%)	DDIM 50 · CFG 전부 기각	0.174917
team8	—	DynamiCrafter_512	액션 cross-attn UNet + 추론 액션 CFG 2.0	배경 고정 후처리 (−0.039)	0.1810 / 0.21621
team7	—	Cosmos-Predict2.5-2B (동결)	자체 심판(IDM)을 만들어 채점 기준을 직접 미분 (DRaFT)	30 step · 오토가이던스 0.55 · 결정적	0.184622

점수는 **낮을수록 좋다**. 표는 각 팀이 문서에 적은 리더보드 값이며 team8만 Public / Private를 함께 남겼다. 순위는 검사보고서에 명시된 팀만 적었다 — team7·team8은 문서에 순위 기재가 없다. team2 · team5 폴더는 비어 있어 분석 대상에서 빠졌다.

01 — 과제와 채점

배점의 40%가 “시킨 대로 움직였는가”다

점수는 세 성분의 가중평균이고 **낮을수록 좋다**. 이 트랙의 이야기는 전부 **Action 성분 (0.4)**에서 시작한다 — 채점기가 생성 영상만 보고 관절 각도를 거꾸로 읽어내 원래 명령과 비교하는 항목이다.

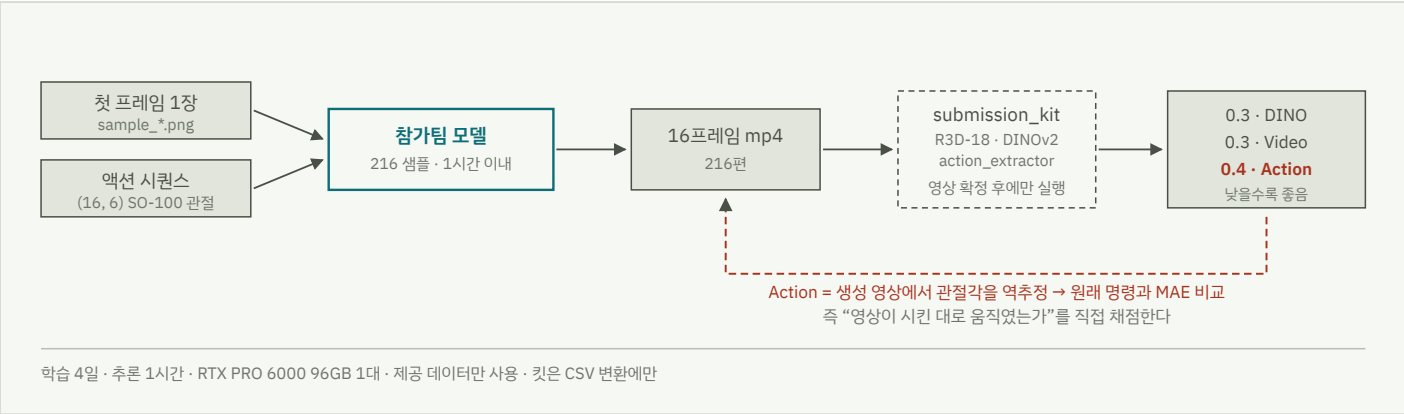


그림 1. 채점 경로. 배점이 가장 큰 항목이 하필 조건 반영도를 재는 항목이라, “모델이 액션을 얼마나 진지하게 듣는가”가 곧 점수가 된다. 이것이 학부 트랙 여섯 팀이 공통으로 붙든 문제다.

02 — 출발 점

네 종류의 백본, 그러나 전략은 하나

백본은 넷으로 갈렸지만 전략은 여섯 팀이 똑같다 — 큰 모델은 통째로 열리고, 얇은 부품만 학습한다. GPU 1장·4일이라는 제약에서 나온 필연적 선택이고, 그래서 팀의 개성은 “무엇을 열렸나”가 아니라 “얼마나 작은 것을, 어디에 붙였나”에서 갈린다.

사전학습 출발점 (전부 동결)	팀 · 새로 붙여 학습한 것	학습 비율
Cosmos3-Nano 36층 · 약 69억 · OpenMDW 2팀 (동일 코드 계열)	team1 · LoRA 504장 + 소모돌 158개 team3 · 같은 구성, rank 1280	6.98 GB 약 23%
Wan2.2 TI2V-5B 30 블록 · 50억	team4 · AdaLN-Zero 주입기 + LoRA r32	15.8M + r32
DynamiCrafter_512 14억 · 대회 baseline 백본 2팀	team6 · 액션 전용 cross-attn 부품 1개 team8 · 액션 cross-attn UNet 4단계 FT	0.08% UNet 부분
Cosmos-Predict2.5-2B	team7 · LoRA r32 280자리 + 액션 임베더	5.7%

그림 2. 오른쪽 열이 이 트랙의 성격을 보여 준다 — 학습 비율이 0.08%에서 23%까지 두 자릿수 배로 벌어진다. team6은 “부품 하나만 바꿔 원인을 특정하겠다”는 실험 설계를 택했고, team1·team3은 반대로 대용량 어댑터로 밀었다.

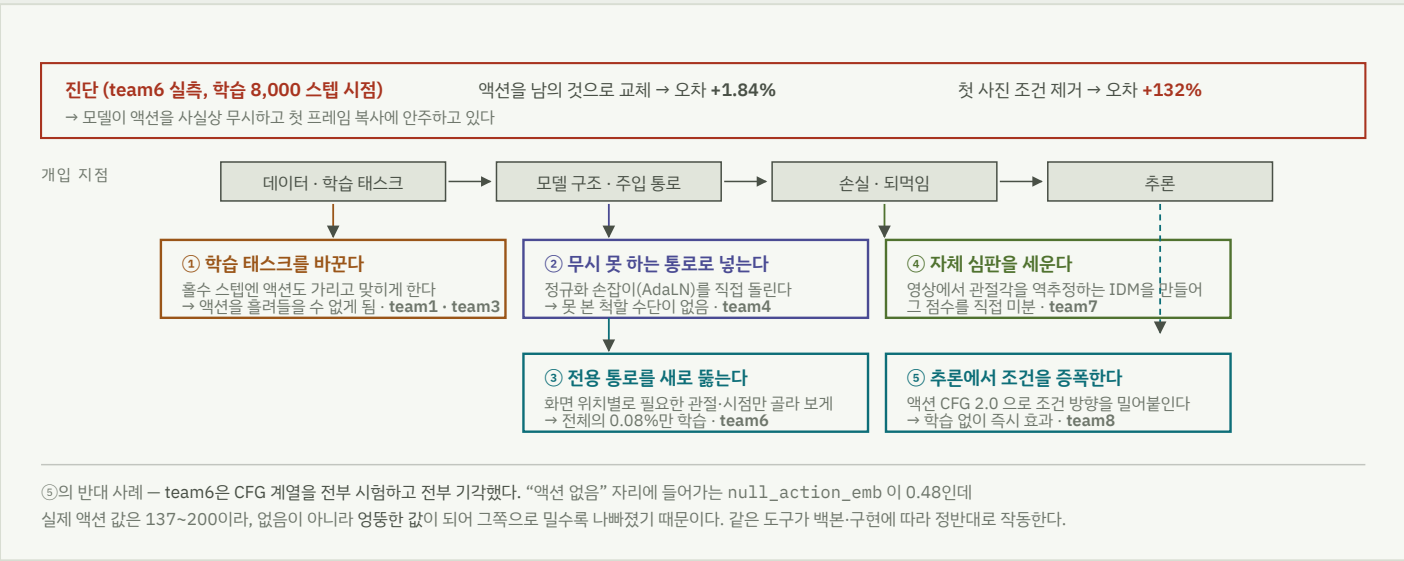
03 — 이 트랙의 중심 문제

“모델이 액션을 무시한다”와 다섯 가지 처방

team6이 학습 도중 측정한 두 숫자가 문제를 정확히 드러낸다. 액션을 남의 것으로 바꿔치기해도 오차는 1.84%만 올랐고, 첫 사진을 빼자 132%가 올랐다. 모델이 액션을

듣는 척하면서 실제로는 첫 프레임을 복사하는 쉬운 답에 안주하고 있었다는 뜻이다.

배점 0.4가 걸린 항목에서 모델이 손을 놓고 있으니, 여섯 팀의 설계는 전부 이 지점을 향한다. 처방은 **학습 태스크 · 주입 통로 · 전용 경로 · 자체 심판 · 추론 증폭**의 다섯 갈래로 갈렸다.



여섯 개의 제출물, 다섯 개의 아이디어

각 팀은 한 줄 아이디어 → 세 줄 요약 → 파이프라인 그림 → 상세 제원 순서로 같은 틀을 쓴다. 그림은 위가 학습, 아래가 추론이다. team1과 team3은 코드가 사실상 같으므로 차이에 초점을 맞춰 나란히 읽는 것이 좋다.

액션을 흘려듣지 못하게, 학습 스텝마다 액션도 맞히는 문제를 번갈아 낸다.

무엇을 Cosmos3-Nano(36층)를 통째로 열리고 LoRA 504장(36층 × 7군데, 6.98 GB)과 소모돌 158개만 학습. 액션은 **18채널**로 부풀린다 — q (지금 자세) · $q - q_0$ (출발점 대비) · Δq (직전 대비).

왜 통했나 두 가지가 겹쳤다. ① **번갈아 내는 문제**(`action_mode = joint`) — 짝수 스텝은 액션을 주고 영상을 맞히게, 홀수 스텝은 액션까지 가리고 둘 다 맞히게 한다. ② **학습 노이즈 분포를 추론 격자에 맞췄다** — 추론이 30 step만 돌므로, 학습 샘플의 70%를 그 30개 지점 위에서만 냈다. 시험에 나올 구간만 집중 연습한 셈이다.

남은 문제 학습·추론에 쓴 GPU가 **CMP 170HX**(채굴용 카드)라 대회 기종과 다르고, 팀 스스로 “카드를 바꾸면 영상도 점수도 달라진다”고 문서에 적었다. 그리고 아래 team3과 코드가 사실상 같다(→ 교차 관찰).

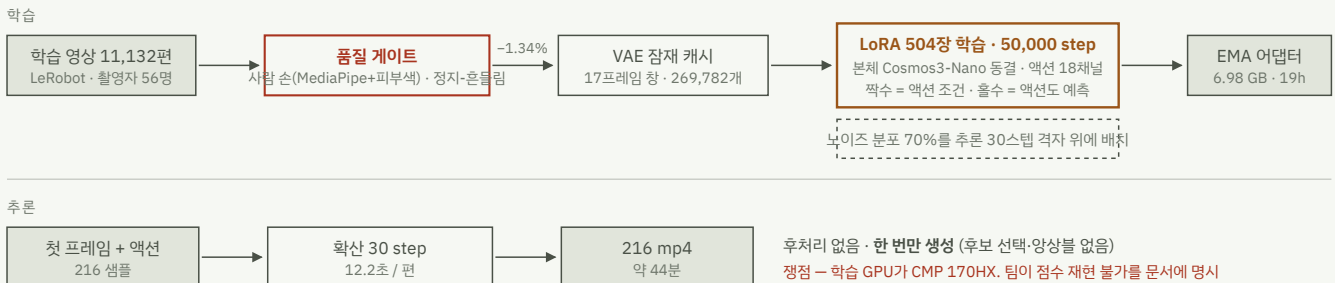


그림 4. team1 — 데이터 청소와 학습 태스크 설계에 힘을 싣고, 추론은 가장 단순하게 끝낸다.

백본	NVIDIA Cosmos3-Nano 36층 (동결) + Wan2.2 VAE (동결)
학습 대상	LoRA 504장 (36층 × 7군데) + 소모돌 158개 · 어댑터 6.98 GB
액션 표현	18채널 = q (6) + $q - q_0$ (6) + Δq (6) · z-score 정규화
학습	50,000 step · 약 19시간 · EMA 제출 · <code>action_mode = joint</code>
데이터	11,132 에피소드 → 품질 게이트 → 273,451 → 269,782 창
추론	30 step · 12.2초/편 · 216편 약 44분 · 후처리 없음

team1과 같은 설계에, 화질 증강과 VLM 캡션을 얻었다.

무엇을	백본·LoRA 구성(504 모듈)·18채널 액션·번갈아 내는 학습 태스크가 team1과 같다. LoRA는 rank 1280 으로 크게 잡아 학습 대상이 34.9억(전체의 약 23%)이고, 60,000 계획 중 50,000 스텝 시점을 제출했다.
차이점	① 학습 창의 절반에 화질 열화 증강(흐림·밝기/대비·노이즈 ↔ 선명)을 창별 고정 시드로 건다 — 평가 이미지 화질이 학습 영상보다 고르지 않다는 관찰에서 나왔다. ② 추론 시 VLM 캡션과 정지 구간 카운트를 조건으로 함께 넣는다. ③ 씬 균등 샘플링(한 장면 2% 상한)과 홀드아웃 24장면.
남은 문제	team1과 코드 구조 유사도 0.783(다른 모든 쌍 0.15–0.26). 홀드아웃 24장면·학습 창 수·하이퍼파라미터 30개가 전부 일치한다. 또 증강 상수가 eval을 실측해 정한 값이라는 주석이 남아 있어 검사보고서가 별도 쟁점으로 올랐다.



그림 5. team3 — 파이프라인의 뼈대는 team1과 같고, 증강과 캡션 조건이 없혀 있다.

백본	NVIDIA Cosmos3-Nano (동결) · 트랜스포머 36층 · 약 150억 규모로 문서 기재
학습 대상	LoRA rank 1280 · 504 모듈 · 34.9억(약 23%) + 소모듈 158개(3,800만)
학습	60,000 계획 중 50,000 step 제출 · lr 8e-5 (액션부 5배) · EMA 25스텝
데이터	약 39만 창 → 3검사 필터 → 269,782 · 홀드아웃 24장면 · 절반에 화질 증강
추론	30 step · VLM 캡션 + 정지 구간 + 18채널 액션 조건
차별점	평가 이미지 화질 편차를 겨냥한 증강 · 씬 균등 샘플링

액션을 못 본 척할 수 없는 통로로 넣고, 움직이는 곳에만 점수를 몰아준다.

무엇을

Wan2.2 TI2V-5B(30 블록, 50억)를 열고 **AdaLN-Zero 액션 주입기 15.8M**과 LoRA r32만 학습. AdaLN은 모델 내부의 정규화 손잡이를 직접 돌리는 방식이라 모델이 무시할 수단이 없고, zero-init이라 학습 첫 순간의 출력은 원본과 완전히 같다.

왜 통했나

손실을 셋으로 나눴다. ① **움직임 가중 flow matching** — 팔이 차지하는 면적이 좁아 그냥 두면 배경만 잘 그리므로, 움직이는 영역에 가중치를 준다(마스크는 압축 전 원본 픽셀에서 계산). ② **REPA** — 12번째 블록의 표현을 DINOv2와 정렬(디코딩 불필요). ③ **미러 손실** — 노이즈가 적을 때($\sigma \leq 0.5$, 실제 발동 17%)만 디코드해 픽셀로 비교.

추론의 한 수

LoRA를 1.05배로 조금 세게 적용하고(리더보드 스왑에서 $1.0 \rightarrow 0.15261$, $1.05 \rightarrow 0.14725$, $1.1 \rightarrow 0.14926$ 의 꼭짓점), 같은 문제를 **시드만 바꿔 5번 생성한 뒤 나머지 4편과 가장 닮은 1편(메도이드)**을 고른다. 정답을 보지 않고 자기 생성물끼리만 비교하는 선택이다. 53.9분으로 60분 제한에 6분 여유.

학습



추론

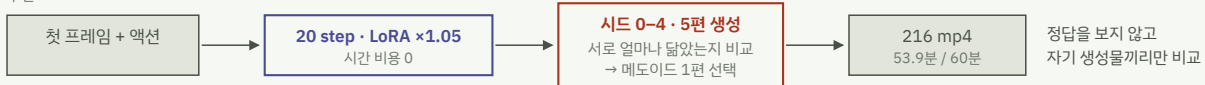


그림 6. team4 — 학습에서 세 손실을, 추론에서 두 요령(LoRA 증폭·메도이드 선택)을 겹쳤다. 추론 시간은 60분 한도에 6분 여유뿐이다.

백본	Wan2.2 TI2V-5B (30 블록 · 50억, 동결)
학습 대상	AdaLN-Zero 액션 주입기 15.8M + LoRA r32
손실	움직임 가중 flow matching + REPA(DINOV2) + 미러($\sigma \leq 0.5$, 발동 17%)
학습	15,000 step · 26.6시간 · 최대 VRAM 78 GB · 학습 데이터 11,132 영상
추론	20 step · LoRA scale 1.05 · K=5 메도이드 선택 · 53.9분
특이	Action 성분 자체 계산에는 킷이 필요해 일부러 쓰지 않았다 — 점수 손해를 감수한 규정 준수 선택

“모델이 액션을 무시한다”를 숫자로 증명하고, 액션 전용 토크 하나만 새로 뚫었다.

무엇을

BottleneckActionCrossAttention — 113만 개(전체 14억의 0.08%)짜리 부품 하나. 관절 6개 × 시점 16개 = **96개 액션 토크**(값 + Δ, 관절·시점 임베딩 부착)을 만들고, UNet middle block의 5×8 = **40개 위치**가 각자 필요한 토크만 cross-attend한다. 기존 통로가 “관절을 한 덩어리로 합쳐 화면 전체에 똑같이 뿌리는” 방식이라 위치별 구분이 안 됐던 문제를 겨냥한다.

왜 통했나

학습을 3단계로 나누고 배우는 범위를 **38% → 38% → 0.08%로 좁혔다**. 마지막 단계에서 나머지를 전부 잠그므로 **바뀐 변수가 딱 하나가** 되어, 좋아지면 그 부품 덕분이라 말할 수 있다. 새 부품에는 **tanh(gate)** 볼륨 손잡이가 0에서 시작해, 켜 직후 출력이 이전 단계와 **비트 단위로 동일**하다.

버린 것

CFG 계열을 여러 방식으로 시험하고 **전부 기각**했다. 액션을 빼면 그 자리에 들어가는 **null_action_emb** 가 0.48인데 실제 액션 값은 137–200이라, “없음”이 아니라 **영똥한 값**이 되기 때문이다. 추가로 baseline 버그 2건(**ddim_eta** 인자명이 조용히 무시됨 · **clean_cond** 가 내부에서 항상 True로 덮어써짐)을 찾아 고쳤다.

학습 — 배우는 범위를 단계마다 좁힌다



데이터 — 손목캠 3개 제외 · 사람 눈 검수(프레임 3,257 표시 / 에피소드 79 제외) · owner 단위 분할로 검증 누수 차단 · 464,631 → 432,090 창

새 부품 내부



추론



그림 7. team6 — 실험 설계가 이 트랙에서 가장 깔끔하다. 마지막 단계에서 바뀐 변수가 하나뿐이라 효과의 원인을 특정할 수 있다.

백본	DynamiCrafter_512 1.4B (대회 baseline 전제 공개 가중치)
신설 모듈	<code>BottleneckActionCrossAttention</code> 1.13M (0.08%) · 헤드 5 × 64차원 · 학습 텐서 12개
학습	stage1 26,000 → stage2 46,000 (5.51억 · 38%) → EMA 접기 → stage3 8,000 (113만 · 0.08%)
데이터	사람 눈 검수 · owner 단위 분할 · 464,631 → 432,090 창 (7.0% 폐기)
추론	DDIM 50 step · <code>clean_cond=False</code> (학습과 정렬) · CFG 미사용
부수 성과	baseline 버그 2건 발견 — <code>ddim_eta</code> 인자 무시 · <code>clean_cond</code> 강제 True

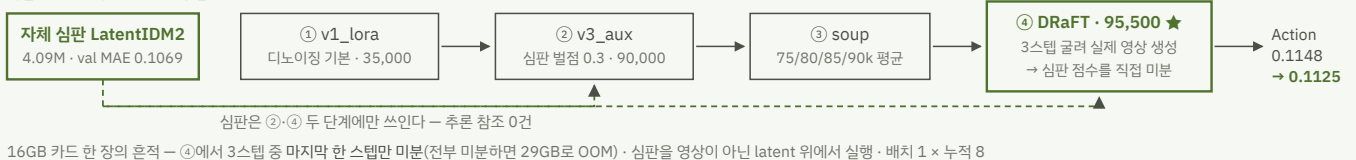
채점기를 흉내 낸 자기 심판을 만들어, 채점 기준을 직접 미분했다.

무엇을 Cosmos-Predict2.5-2B(28 블록)를 열고 LoRA r32 280자리 + 액션 임베더 = **113.4M(5.7%)**만 학습. 여기에 LatentIDM2 (4.09M)라는 **학습 전용 심판**을 대회 데이터만으로 따로 만들었다 — 영상 요약본을 보고 관절각 16×6을 역추정하며, 검증 MAE 0.1069이다. **추론에는 참조가 0건**이다.

왜 통했나 4단계로 가르치는 방법을 바꿔 갔다. ① 기본 디노이징 → ② 심판 벌점 추가(30% 확률, 가중 0.3) → ③ 체크포인트 4개 평균(soup) → ④ **DRaFT** — 3스텝을 실제로 굴려 진짜 영상을 만든 뒤 심판 점수를 직접 미분한다. Action 향이 **0.1148 → 0.1125**로 실제로 내려갔다.

시간 정렬 latent는 시간이 4배 압축돼 4장뿐인데 명령은 16줄이다. team7은 이를 **4줄씩 네 묶음으로 접어** latent 각 장에 째지었다 — “16박자 안무표를 통째로 주는 대신 4박자씩 끊어 해당 장면에 붙이는” 방식이다. 추론은 결정적이라(후보 선택·시드 평균·양상불 전부 없음) 재실행 시 CSV의 md5까지 일치한다.

학습 — 4단계, 가르치는 방법이 바뀐다



추론



그림 8. team7 — 채점 기준을 자체 모델로 복제해 학습 손실에 직접 넣은 접근. 0.91 epoch만 돌고 멈춘 것도 측정에 근거한 판단이다.

백본	Cosmos-Predict2.5-2B (28 블록, 동결) + Wan2.1 VAE (동결) · 빈 텍스트 임베딩 고정
학습 대상	LoRA r32 280자리 + 액션 임베더 = 113.4M (약 5.7%)
자체 심판	LatentIDM2 4.09M · 25,000 step · EMA 0.999 · 검증 MAE 0.1069 · 추론 미사용
학습	① 35,000 → ② 90,000 → ③ soup → ④ 95,500 노출 (0.91 epoch) · RTX 5070 Ti 16GB 1장
추론	UniPC 30 step · shift 3.0 · 오토가이던스 0.55 · β 1.08 · seed 7 · 완전 결정적
데이터	기여자 56명 · 에피소드 10,735 → 17프레임 창 104,828 · latent 사전 캐시(약 12시간)

학습으로 못 다 채운 것을 추론에서 채운다 – 액션 CFG로 밀고, 배경은 얼린다.

무엇을

DynamiCrafter_512에 **액션 cross-attention**을 붙이고 12차원 액션(절대 6 + Δ 6)을 `action_ctx_tokens 16 / action_txt_tokens 32` 로 넣는다. 4단계로 누적 **10,800 step**(300 / 2,100 / 1,800 / 6,600) 학습하며, 단계 전환 시 **UNet 가중치만** 물려받는다 – 옵티마이저는 초기화되고 워밍업을 다시 하므로 한 번에 10,800을 도는 것과 결과가 다르다.

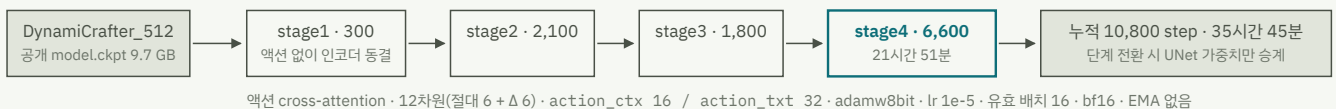
추론이 절반

이 팀 점수의 상당 부분이 추론에서 나온다. **배경 고정 후처리만으로 0.2198 → 0.1810**(-0.039)이고, 첫 프레임 16회 반복이라는 정적 기준선이 0.3019임을 감안하면 모델의 기여와 후처리의 기여가 모두 큰 구성이다. 여기에 액션 CFG 2.0을 걸었다 – team6이 기각한 바로 그 도구다.

남은 문제

순위에 쓰인 2026-08-19 학습의 **로그와 체크포인트가 삭제**됐다. 대신 팀이 **같은 코드·설정·시드로 전 구간 재학습**을 돌려 Public 0.1810→0.1886, Private 0.2162→0.2239로 두 지표가 거의 같은 **폭(+0.0077)으로 움직임**을 보였다. 추론 시간은 59분 54초로 한계선에 붙어 있다.

학습 — 4단계 누적 10,800 step



추론 — 여기서 점수가 크게 움직인다

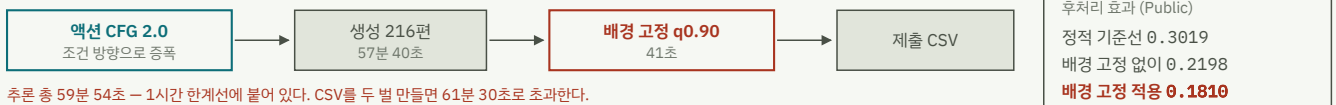


그림 9. team8 — 유일하게 후처리 기여를 숫자로 분리해 보여 준 팀이다. 배경 고정 하나가 -0.039를 만든다.

백본	DynamiCrafter_512 (공개 model.ckpt 9.7 GB) · 액션 cross-attention UNet
액션	12차원 (절대 6 + Δ 6) · action_ctx_tokens 16 / action_txt_tokens 32
학습	4단계 누적 10,800 step · adamw8bit · lr 1e-5 · 유효 배치 16 · bf16 · 35시간 45분 · seed 20230211
추론	액션 CFG 2.0 + 배경 고정 q0.90 · 59분 54초 · seed 0
후처리 효과	정적 기준선 0.3019 → raw 0.2198 → bg 0.1810
재학습 검증	Public 0.1810→0.1886 · Private 0.21621→0.2239 (두 지표 +0.0077 동일 폭)

여섯 제출물을 나란히 놓고 보면

team1과 team3은 사실상 하나의 코드다

함수·변수 이름을 전부 지우고 구조만 비교한 유사도에서 **team1↔team3만 0.783**이 나온다. 다른 모든 쌍은 0.15–0.26이고, 12자 넘는 동일 문자열도 team1↔team3은 404개인 반면 나머지 쌍은 4–22개다.

팀 쌍별 코드 구조 유사도 (이름 제거 후 비교 · 1.0 = 동일)



코드 밖 일치 — 홀드아웃 같은 24장면 · 학습 창 269,782 동일 · 하이퍼파라미터 30개 전부 일치 · 동봉 손검출 모델 sha256 동일 · 가중치 파일은 서로 다름

그림 10. 유사도 분포. team1↔team3 한 쌍만 다른 쌍들의 4–5배다. 가중치 파일 자체는 서로 다르므로 파일 복사가 아니라 같은 코드를 각자 한 번씩 돌린 것으로 보인다. 어느 쪽이 원본인지는 이 자료로 판단할 수 없다.

같은 도구가 팀에 따라 정반대로 작동했다

이 트랙에서 가장 배울 점이 있는 대목이다. **액션 CFG**는 team8에서는 최종 레시피에 들어갔고(2.0), team6에서는 전부 기각됐다. team6이 밝힌 이유가 명확하다 — “액션 없음” 자리에 들어가는 `null_action_emb` 가 0.48인데 실제 액션 값은 137–200이라, 그 방향으로 미는 것이 조건 없는 쪽이 아니라 엉뚱한 값 쪽으로 미는 일이 되기 때문이다. team7도 액션 CFG를 재 보고 기각했다(오토가이던스와 서로 방해). **같은 이름의 기법이라도 액션 임베딩의 스케일 설계에 따라 효과가 뒤집힌다.**

학습과 추론의 조건을 맞추는 것이 반복해서 등장한다

- **team1 · team3** — 추론이 30 step만 도니, 학습 노이즈 분포의 70%를 그 30개 지점 위에 몰았다.
- **team6** — 학습에 첫 프레임 앵커가 없었으므로 추론에서도 `clean_cond=False` 로 두어 첫 프레임에 같은 노이즈를 걸었다.
- **team4** — 시험 때 액션이 16줄이므로 학습에서도 16줄만 쓰고 17번째 칸은 마지막 줄을 복사했다.
- **team7** — latent 4장에 액션 16줄을 4줄씩 접어 시간축을 명시적으로 맞췄다.

데이터 청소가 절반이었다

여섯 팀 중 넷이 학습 데이터를 그대로 쓰지 않았다. team1·team3은 MediaPipe 손 검출과 피부색·모션 검사로 창을 걸렀고(269,782), team6은 **사람이 직접 눈으로 검수**한 뒤 손목캠을 빼고 **owner 단위로 분할**해 검증 누수를 막았으며(432,090), team7은 자체 IDM으로 데이터를 채점해 에피소드 3.0%를 덜어냈다. “팔은 멈췄는데 화면만 흔들리는 구간”을 배우면 모델이 액션과 무관하게 화면을 흔들도록 학습된다는, 이 트랙의 중심 문제와 직결된 처리다.

기록 상태

- **team1** — 학습 GPU가 CMP 170HX(대회 기종과 다름). 팀이 점수 재현 불가를 문서에 명시.
- **team8** — 순위에 쓰인 학습의 로그·체크포인트 삭제. 대신 전 구간 재학습으로 Public·Private가 같은 폭으로 움직임을 증명.
- **team7** — 같은 레시피 재학습 시 ± 0.03 분산. 점수 재현은 동봉 가중치로만 가능하다고 팀이 명시.
- **team3** — 증강 상수가 eval 실측에서 유래했다는 주석이 남아 있고, team1 코드에도 같은 상수가 있다.

이 문서의 범위. 제출 README·config·소스코드, 기존 검사보고서, 그리고 `undergrad/diagram/`의 방법론 그림 문서를 읽어 **설계 아이디어만** 정리했다. 규정 위반 여부의 판단이 아니며, “쟁점”으로 표시한 항목은 기존 검사보고서의 지적을 맥락으로 옮겨 적은 것이다. 코드는 읽기만 했고 실행하지 않았으며, 제출물은 수정하지 않았다.